



Thetanuts Hackathon — Builder Resources

Thetanuts Finance V4 is a decentralized options protocol live on **Base mainnet**. This is a kickstart sheet, not a manual — it points you at the right resource and gets out of the way.

Full documentation:

- docs.thetanuts.finance/for-builders/sdk
- docs.thetanuts.finance/sdk

Source of truth: github.com/Thetanuts-Finance/thetanuts-sdk

If anything here disagrees with the repo or the docs, they win — and please open an issue so we fix it.

More below!



Part 1 — The Two Tracks

Track 1 — Best Product Built on the Thetanuts SDK

Build any working product on top of the Thetanuts SDK. Structured products, vaults, consumer trading apps, options-powered lending, analytics tools, or something we haven't thought of. Open-ended: bring us a real, working product that uses on-chain options in a meaningful way.

Track 2 — AI × Options · 1000 USDC

Build an AI agent that places a real on-chain options trade on Thetanuts' OptionBook, live on Base mainnet. Think natural-language trading, an AI strategy/risk copilot, or an autonomous hedging agent. The bar: your agent must execute **at least one real trade against our live pricing** — not paper trading, not testnet.

What we're judging

Both tracks, three things:

1. **Does it work?** A real, running product — not a mockup, not a README describing what you would have built
2. **Are the options load-bearing?** If it would work identically with the Thetanuts calls stubbed out, it isn't really using on-chain options.
3. **Does it fit the market?** Who is this for, why would they use it over what already exists, and what happens after the hackathon? We're as interested in a product that could find real users as in a clever technical demo. Tell us who you're building for and why they'd care — a couple of honest sentences beats a business plan.



Part 2 — Resources

Start here: app.thetanuts.finance/tools — every tool in one place. For anything deeper, follow the GitHub links below.

Tool	Version	What it is	Link
SDK <code>@thetanuts-finance/thetanuts-client</code>	0.3.0	The main event. TypeScript client, ESM + CJS + full types, Node 18+ and browsers. Everything else is a layer on top of this.	github.com/Thetanuts-Finance/thetanuts-sdk
CLI <code>@thetanuts-finance/cli</code>	0.5.0	Terminal access to the protocol. <code>-o json</code> makes it a scriptable API for agents that would rather shell out than import a library. Binary is <code>thetanuts</code> .	github.com/Thetanuts-Finance/thetanuts-sdk/tree/main/cli
MCP server <code>@thetanuts-finance/mcp</code>	1.0.0	~100 tools exposing the protocol to any LLM over Model Context Protocol. Reads state, runs pricing math, builds transactions . Never signs, holds no keys.	github.com/Thetanuts-Finance/thetanuts-sdk/tree/main/mcp-server

For reference — odette.fi. A live on-chain daily options platform running on Thetanuts infrastructure: 1–3 day expiries, up to 2000x leverage, no liquidation risk. Worth a look to see what a production integration is capable of. Not a template, and not something to copy for either track.

Get an RPC first

`https://mainnet.base.org` is the public Base endpoint. It works, but it's rate-limited and it will start throttling you the moment your agent polls in a loop or your indexer backfills. You'll lose an hour to mystery timeouts that look like bugs in your code.



Grab a **free** key before you write anything — 30 seconds, no card:

- **Alchemy** — alchemy.com → create app → Base Mainnet
- **Infura** — infura.io → create key → Base

Then use it everywhere:

```
export  
THETANUTS_RPC_URL="https://base-mainnet.g.alchemy.com/v2/YOUR_KEY"
```

The SDK takes any ethers provider, and the CLI reads `THETANUTS_RPC_URL` or `--rpc-url`.

30-second setup check

No wallet, no signer, no approvals. If this prints live prices, you're connected to the real protocol.

```
import { ethers } from 'ethers'; import { ThetanutsClient } from  
'@thetanuts-finance/thetanuts-client'; const client = new  
ThetanutsClient({ chainId: 8453, provider: new  
ethers.JsonRpcProvider(process.env.THETANUTS_RPC_URL), });  
console.log((await client.api.fetchOrders()).length, 'live orders');  
console.log(await client.api.getMarketData());
```

Everything past this point — filling, RFQs, positions, pricing — is in the docs and the repo README. We're not duplicating it here.

OptionBook vs RFQ

Two ways to get an option. Both produce identical positions using the same cash-settled implementation contracts — the difference is how you get a quote.

	OptionBook	RFQ (OptionFactory)
What	Fill existing market-maker orders from a public book	Request a custom option; MMs submit sealed-bid offers
Best for	Quick trades on listed strikes/expiries	Custom strikes, custom expiries, off-the-run structures
Structures	Vanilla, spread, butterfly, condor, iron condor	Same, plus optional physically-settled vanilla
Settlement	Cash-settled only	Cash-settled by default; physical is opt-in, vanilla only



Liquidity	Already on the book — fill it	A live market maker answers your request
-----------	-------------------------------	--

Rule of thumb: if the order you want already exists on the book, fill it. If not, RFQ it. A market maker is live and quoting on both, so either path works during the hackathon.

Part 3 — Before You Trade

This is mainnet with real funds. The SDK's chain config defines only Base mainnet (8453) and Ethereum mainnet (1) — there is no testnet configuration.

- **Trade small. We mean it.** We're not asking anyone to put size on — the point is proving your code executes a real trade, not the notional. A couple of USDC per fill is plenty, and a 1 USDC fill scores exactly the same as a 100 USDC fill. (You can even try with <1 USDC trades)
- **1–3 USDC on Base covers you.** Bring your own — bridge to Base, or withdraw from any exchange that supports Base. Keep a little ETH for gas: cents, not dollars.
- **Use a fresh, disposable wallet.** Never point a hackathon agent at a wallet holding anything you'd miss. `thetanuts wallet create` generates one with locked-down file permissions.
- **Never commit a private key.** Use `THETANUTS_PRIVATE_KEY` or a `.env`, and check your git history before you submit.
- **Approve only what you need** — the exact amount, not `MaxUint256`.
- **Dry-run first.** `--dry-run` is a global CLI flag on every command. In the SDK, preview before you fill.

Getting help

- **Telegram:** @ShawnSeanC — for anything blocking
- **Discord:** the **Thetanuts Finance** chatroom inside the MUBA Hackathon server
- **Bugs, broken behaviour, missing docs:**
github.com/Thetanuts-Finance/thetanuts-sdk/issues — file it, we read them. During the hackathon this is the fastest path to a fix.



Figuring out the protocol

We've deliberately kept the protocol details out of this sheet — they'd go stale, and there are three better places to get them:

1. **Install the MCP server and ask it.** `get_sdk_context` loads the entire SDK — every module, the key types, the common workflows, and the gotchas that trip people up. One question beats reading the whole GitBook.
2. docs.thetanuts.finance/for-builders/sdk — the written version of the same thing.

If your coding agent starts inventing method names, it hasn't loaded the SDK context. Feed it `llms-full.txt` from the repo root and it stops (Check the docs)

Quick reference

Chain: Base mainnet, chainId 8453

RPC: free Alchemy or Infura key (not the public endpoint)

SDK: `npm i @thetanuts-finance/thetanuts-client ethers`

CLI: `npm i -g @thetanuts-finance/cli` (binary: `thetanuts`)

MCP: `npx -y @thetanuts-finance/mcp`

Tools: <https://app.thetanuts.finance/tools>

Docs: <https://docs.thetanuts.finance/for-builders/sdk>

Repo: <https://github.com/Thetanuts-Finance/thetanuts-sdk>

Telegram: @ShawnSeanC